
Definição do trabalho da M2

Data de entrega: 10/05/2026. (até 23:59)

Modalidade: Cinco Integrantes.

Visão Geral:



O Mastermind (no Brasil Senha) é um jogo de tabuleiro inventado por Mordechai Meirowitz e distribuído inicialmente pela Invicta Plastics. Publicado em 1971, o jogo vendeu mais de 50 milhões de tabuleiros em 80 países, tornando-se o mais bem sucedido novo jogo da década de 1970. Atualmente, no Brasil é vendido pela Grow com o tabuleiro preto e cinza, e os pinos do jogo em azul, amarelo, verde, vermelho, rosa, roxo e laranja (Wikipedia).

Descrição:

Um jogo de Mastermind tem pinos de sete cores diferentes, aleatórias, exceto preto e branco. Os pinos pretos e brancos são menores. Há quatro buracos grandes em cada fileira, em 10 fileiras, uma abaixo da outra. E ao lado delas, um quadrado menor, com quatro buracos menores, dois em cima de dois. Uma fileira, que seria a décima primeira, tem um defletor que esconde seus buracos. O desafiador faz uma combinação com quatro pinos coloridos, sem repetir as cores de cada pino, e as põe na décima primeira fileira e levanta o defletor, escondendo a senha. Então, o desafiado tenta adivinhar a senha, pondo quatro pinos que ele acha que são a senha na primeira fileira, e o desafiador põe os pinos pretos e brancos no quadrado menor ao lado. A regra dos pinos pretos e brancos são essas: o branco significa que há uma cor certa mas lugar errado, o preto significa que há uma cor certa no lugar certo, e nenhum pino significa que uma das cores não é contida na senha. O desafiado vai tentando adivinhar, se guiando pelos pinos pretos e brancos. Se o desafiado não acertar até a 10ª fileira, o desafiador fecha o defletor e revela a senha, mas se adivinhar, o desafiador põe quatro pinos pretos e revela a senha.

REGRAS PARA O DESENVOLVIMENTO

O jogo deverá possuir um menu onde será possível escolher:

- Jogar
- Sobre
- Sair

A seguir serão listadas, de trás para frente, o que deve ser implementado em cada parte do menu.

Sair:

Essa é uma opção para finalizar o programa. Observe que seu jogo só deve ser encerrado ao selecionar essa opção, caso qualquer outra opção seja escolhida ela deve retornar ao menu no fim de sua execução.

Sobre:

Quando essa opção for selecionada, deverão ser exibidas a equipe de desenvolvimento (o nome de cada membro da equipe), o mês/ano e o nome do professor/disciplina.

Jogar:

Essa é a parte onde o jogo acontece de verdade!

O jogador terá que acertar uma sequência de 4 dígitos e terá 10 tentativas para isso.

Inicie o jogo gerando os números aleatórios que serão o código de 4 dígitos que o jogador tentará quebrar. Esse código deverá respeitar as seguintes regras:

- Não deverão ser gerados números repetidos; e
- Os números gerados deverão estar entre 1 e 6.

Uma vez escolhido o código que deverá ser descoberto pelo jogador, você deverá solicitar que o jogador digite um código (4 dígitos)

Após cada tentativa o jogador deverá ser informado:

- O número de tentativas restantes
- Quantos foram os números digitados que estão corretos e estão na posição correta. (note que você diz quantos e não quais)
- Quantos foram os números digitados que estão corretos, mas não estão nas posições corretas.

O jogo deve acabar quando o jogador acertar todos os números na ordem correta ou quando o número de tentativas zerar.

Se o jogador vencer, informe que ele venceu e retorne ao menu.

Se o jogador perder, informe que ele perdeu e retorne ao menu.

Dicas de desenvolvimento:

O código, a seguir, exemplifica o uso das funções `rand()` e `srand()`;

- `rand()` gera um número pseudo-aleatório entre 0 e `RAND_MAX`, mas essa faixa pode ser facilmente alterada com o operador de resto da divisão inteira.
- `srand()` gera uma nova semente aleatória baseada no parâmetro passado entre os parênteses da função. É comum utilizar a função `time()`, pois ela pega o horário do sistema que muda a cada milésimo de segundo. Note que se a função `srand()` não for utilizada a sequência de números pseudo-aleatórios gerados pela função `rand()` será sempre a mesma.

```

1  #include <iostream>
2  using namespace std;
3  #include <time.h> //Para habilitar a função time
4  #include <stdlib.h> //Para habilitar o comando srand
5
6  int main() {
7
8      srand(time(NULL)); //semente randomica gerada a partir da hora do sistema
9
10     int numeroAleatorio;
11
12     numeroAleatorio = rand()%10; //0 %(mod) coloca os números gerados entre 0 e o resto da divisão -1
13
14     cout << numeroAleatorio << endl;
15
16 }

```

Outros dois comandos bastante úteis no desenvolvimento de programas no console, são os comandos `system("cls")` e `system("pause")`.

- `system("cls")` é um comando que limpa a tela do console (**c**lear **s**creen). Esse comando é bastante útil, pois em uma tela limpa é mais fácil dar destaque aquilo que se está mostrando no momento.
- `system("pause")` é um comando útil, principalmente quando usado em conjunto com o `system("cls")`, pois ele pausa a execução da aplicação até que o usuário aperte qualquer tecla, bastante útil quando se quer exibir algo antes de limpar a tela para iniciar uma nova execução.

*Os comandos alternativos ao `system("cls")` e `system("pause")` são respectivamente o `cout<<"\033c"` e `cin.ignore()`.

Obs.: Para o desenvolvimento do código não poderão ser utilizados comandos não abordados na disciplina, variáveis compostas (*arrays*) e funções.

Defesa (Obrigatória)

Durante a defesa serão realizados questionamento sobre o trabalho realizado pelo grupo. A defesa é obrigatória e deverá ser feita pelos integrantes do grupo na aula. Se algum integrante não estiver presente durante a aula de defesa, deverá entrar com um pedido de segunda chamada, o mesmo defenderá posteriormente em data a ser agendada com o professor.

Entregas:

- *Postar nesta atividade: **Trabalho T2***
- *Código fonte desenvolvido (.cpp ou .txt): é de responsabilidade do grupo verificar se o arquivo postado é o correto.*

Critérios de Avaliação:

- 1. Organização e clareza do código = 5% da nota.*
- 2. Identificação dos autores e Comentários pertinentes e oportunos no código = 10% da nota.*
- 3. Funcionamento correto conforme a especificação = 40% da nota.*
- 4. Recursos da linguagem utilizados = 20% da nota.*
- 5. Apresentação/Defesa do código = 25% da nota.*

Obs.: Todas as notas relativas ao código dependem do desempenho na defesa. Sem a defesa o trabalho terá nota ZERO.